AI Compiler Performance Engineering

Auto-Loops Books — ch08

Contents

1 TMA Double-Buffer Pipelines and Async Movement **1**

1.1 Pipeline theory 1

1.2 GPU TMA 3

1.3 CPU DMA 5

1.4 NPU pipeline 7

1.5 YiRage pipeline generation 8

1.6 Key Takeaways 10

1.7 Conclusion 11

Bibliography **12**

Index **14**

Chapter 1

TMA Double-Buffer Pipelines and Async Movement

Chapter 7 fixed roles so the consumer units never wait on a scheduler; this chapter makes sure they never wait on data either, by pipelining asynchronous movement so the next tile is already arriving while the current one is consumed (Shah *et al.*, 2024; SemiAnalysis, 2024). The stakes are the decode metrics established in Chapter 1: at batch-1 the model re-reads weights and the growing KV cache every token, so the load path, not the matmul, sets ms/token, and any cycle a compute unit spends idle waiting for a load is a cycle of pure latency (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Dense Llama-3.2-1B already issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305; collapsing those launches only pays off if the resulting fused kernel keeps its compute units fed (Chen, 2026; NVIDIA, 2024). The mistake to avoid is treating data movement as synchronous—load, then compute, then load. That serializes the two and exposes the full memory latency every step. The fix, across every hardware class, is a **double-buffer pipeline**: overlap the movement of tile $n{+}1$ with the computation on tile n, so latency is hidden behind useful work (Dao *et al.*, 2022; AMD, 2024a).

1.1 Pipeline theory

The theory of a software **pipeline** is simple and general: split a stream of work into stages, give each stage its own buffer, and run the stages concurrently so that while one stage consumes buffer A, the previous stage fills buffer B (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). The idea is borrowed directly from hardware instruction



Figure 1.1: A double-buffer pipeline: after a one-tile prologue, each compute step overlaps with the load of the next tile, hiding memory latency behind compute.

pipelines and from classical producer-consumer queueing, and the same intuition applies: throughput is set by the slowest stage, and latency is hidden as long as there is buffered work for the consumer to do while the producer runs ahead.

The reason a **pipeline** is the right frame for decode specifically is that decode is a stream of nearly identical, small units of work—one token's layer computation after another, each reading a predictable working set (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Streaming, regular work is the ideal case for pipelining: the stages are stable, the buffer sizes are known, and the depth can be fixed at compile time. This is in contrast to irregular, data-dependent work where the next tile's size or location is not known until the current one finishes, which is harder to pipeline and is one reason prefill and decode want different schedules (Dao *et al.*, 2023; Chen, 2026).

Double-buffering is the two-stage instance—one buffer being computed on, one being loaded—and it is the minimum needed to overlap movement with compute. The cost is doubled buffer footprint, which is exactly the residency charge Chapter 6 told the planner to account for; the benefit is that, after a one-tile prologue, the memory latency of every subsequent load is hidden behind the compute of the previous tile (SemiAnalysis, 2024; Chen, 2026).

Two quantities govern a **pipeline**'s effectiveness. The first is depth: a deeper **pipeline**, with more than two buffers in flight, hides more latency because more loads can be outstanding, but it consumes proportionally more fast memory (Shah *et al.*, 2024). The second is the balance between the stages: if the load stage is slower than the compute stage, the **pipeline** is memory-bound and stalls no matter how deep it is, and if compute is slower, the loads complete early and the extra depth is wasted. For batch-1 decode the load stage usually dominates—the matmul is a thin matrix-vector product while the load is a full weight tile—so the design goal is to make the load as asynchronous and as overlapped as the hardware allows, then size the **pipeline** depth to the available fast memory (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

A worked latency argument shows why depth matters. Suppose a tile load takes 400 cycles of memory latency and the compute on a tile takes 300 cycles. With no pipeline, each step costs $400 + 300 = 700$ cycles, and the unit is idle 57%

of the time. With a two-stage **double-buffer**, the steady-state step cost drops to $\max(400, 300) = 400$ cycles—the load and compute overlap, so the step is bounded by the slower stage—and the memory latency is fully hidden behind compute only if compute were the longer stage (SemiAnalysis, 2024; Dao *et al.*, 2022). Here the load is longer, so the **pipeline** is memory-bound at 400 cycles per tile, and adding a third buffer does not help because the bottleneck is load bandwidth, not the number of outstanding loads. This is the quantitative form of the balance rule: deepen the **pipeline** only until the bottleneck stage saturates, then stop.

The same arithmetic explains a counter-intuitive decode result. Because decode is memory-bound, the best a **pipeline** can do is hide the compute behind the load, which means the achievable token latency is bounded below by the time to stream the per-token working set—the weights and KV slice—no matter how clever the overlap (Vellaisamy *et al.*, 2026; Chen, 2026). Pipelining cannot beat that floor; it can only ensure the kernel reaches it instead of paying load-plus-compute serially. This is why Chapter 6's residency work and this chapter's movement work are complementary: residency reduces how many bytes must be streamed, and pipelining ensures the bytes that are streamed overlap with compute, and only together do they approach the memory-bandwidth floor (Kwon *et al.*, 2023; Davies *et al.*, 2025).

The crucial enabling property is asynchrony. A **pipeline** only overlaps if the load can be issued and then proceed in the background while the issuing unit moves on to compute; a synchronous copy that blocks until it completes cannot overlap with anything (SemiAnalysis, 2024). Every hardware class therefore provides some asynchronous movement primitive, and the chapters that follow are really a tour of those primitives—TMA on a GPU, prefetch and async copy on a CPU, DMA on an NPU—and of how a compiler builds a correct **double-buffer pipeline** on top of each. The common requirement is a completion signal: the consumer must know when buffer B is full before it reads it, so each primitive pairs with a barrier or token that the **pipeline** uses to sequence the stages (Shah *et al.*, 2024; AMD, 2024a).

1.2 GPU TMA

On a **Hopper** GPU the asynchronous movement primitive is the Tensor Memory Accelerator (**TMA**), and it is purpose-built for the **double-buffer pipeline** this chapter describes. **TMA** performs asynchronous bulk copies between global and shared memory, and crucially it is issued by a single thread rather than by every thread in a warp, which frees the other threads' registers and address-generation

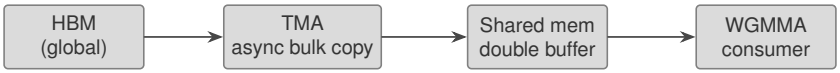


Figure 1.2: On Hopper, TMA issues asynchronous bulk copies from HBM into a double-buffered shared-memory region that WGMMA consumers drain.

work for compute (Shah *et al.*, 2024). This is why FlashAttention-3 assigns **TMA** to a dedicated producer warp: one thread kicks off the bulk load, the producer moves on, and the consumer warpgroup runs WGMMA on the previously loaded tile (Shah *et al.*, 2024).

Completion is signaled through transaction barriers. A **TMA** copy is associated with a shared-memory barrier that tracks how many bytes have arrived, and the consumer waits on that barrier before reading the buffer; because the barrier is transaction-based, the hardware knows when the full tile has landed rather than relying on a thread count (Shah *et al.*, 2024; NVIDIA, 2022). This is the completion signal the **pipeline** theory required, and it is what lets the compiler build a correct multi-stage pipeline: each buffer has its own barrier, the producer fills buffer $n{+}1$ and arrives at its barrier while the consumer waits on buffer n's barrier, and the two proceed in lockstep one tile apart.

TMA also carries optimizations that matter for decode residency. It can multicast a single load to the shared memory of all CTAs in a thread-block cluster in one operation, so a weight tile read once is delivered to several consumers without re-reading HBM—a direct bytes/token saving when the same weights feed multiple CTAs (Shah *et al.*, 2024; NVIDIA, 2022). It also exposes an L2 promotion hint that biases the cache toward keeping **TMA**-loaded lines resident when the same input tile is consumed by many CTAs, as in a GEMM where each weight column feeds every row (SemiAnalysis, 2024; NVIDIA, 2022). Both are mechanisms the **pipeline** builder uses to ensure that the bytes a tile needs are either already on chip or arriving asynchronously, never fetched synchronously on the critical path.

There is a register dimension to **TMA** that is easy to miss and directly relevant to the residency budget. On older async-copy paths every thread in a warp participated in address generation, consuming registers and instruction slots that then could not be used for compute (Dao *et al.*, 2022). Because **TMA** is issued by a single thread with a hardware-resident multidimensional descriptor, it frees the rest of the warp's registers, and combined with Hopper's warpgroup-wide register reallocation from Chapter 7 this lets the consumer warpgroup claim a larger share of the 65,536-register file for accumulators (Shah *et al.*, 2024; NVIDIA, 2022). The movement primitive and the residency budget are thus coupled: choosing **TMA** is not only a movement decision but a register decision, and a **pipeline** built on it

can sustain deeper compute tiles than one built on the older per-thread copy.

The interaction with the transaction barrier is what makes deep **TMA** pipelines correct rather than merely fast. Each buffer in the **pipeline** gets its own barrier object in shared memory; the producer thread issues the **TMA** for buffer k and records the expected byte count on that buffer's barrier, and the consumer warpgroup waits on barrier k before reading buffer k (Shah *et al.*, 2024). With s buffers the producer can run up to s tiles ahead, and the barriers form a circular list that sequences the stages—a structure CUTLASS encapsulates in reusable pipeline classes precisely because managing the barriers by hand is error-prone (Shah *et al.*, 2024; NVIDIA, 2022). For a compiler this is ideal: the barrier discipline is mechanical and can be generated, which is exactly what the YiRage **pipeline** pass does.

The predecessor primitive, `cp.async`, shipped on Ampere and established the pattern **TMA** perfected: asynchronously copy from global to shared memory and commit groups of copies that the consumer can wait on (Dao *et al.*, 2022). **TMA** improves on it by handling multidimensional tile descriptors in hardware, issuing from one thread, and integrating with the transaction barrier, which is why the deepest decode pipelines target **Hopper**'s **TMA** path (Shah *et al.*, 2024; SemiAnalysis, 2024).

A practical caution applies when reasoning about **TMA** for decode rather than prefill. **TMA**'s headline benefits—bulk multidimensional copies, multicast across a cluster, L2 promotion for reused inputs—are largest when the same tile feeds many consumers, the GEMM-like sharing pattern of prefill (SemiAnalysis, 2024; NVIDIA, 2022). At batch-1 decode the reuse is lower: a weight tile is read once per token by a thin matrix-vector product, so the multicast and L2-promotion wins shrink and the dominant benefit reduces to the plain asynchrony—issuing the load early and overlapping it (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The design lesson is to use **TMA** for what decode actually needs (cheap asynchronous loads that free registers) without assuming the prefill-shaped sharing wins transfer, which is another instance of the book's theme that prefill intuitions mislead at the decode operating point.

1.3 CPU DMA

A **cpu** has no programmer-managed scratchpad and no **TMA**, so its version of the **double-buffer pipeline** is built from prefetching and the cache hierarchy rather than explicit asynchronous copies into shared memory (Goodfellow *et al.*, 2016). The hardware prefetcher already overlaps movement with compute oppor-

tunistically: on a detected sequential access pattern it pulls the next cache lines ahead of demand, which for a streaming weight read is effectively a **pipeline** the hardware builds for free (Goodfellow *et al.*, 2016). The kernel's job is to make the access pattern predictable enough for the prefetcher to track and to block the computation so the working set of each tile stays cache-resident across its reuse, as Chapter 6 described.

When the hardware prefetcher is insufficient, software prefetch instructions let the kernel issue an explicit hint to begin loading a cache line some distance ahead of its use, which is the **cpu** analogue of issuing an asynchronous load early in a **pipeline** (Goodfellow *et al.*, 2016). The distance—how many iterations ahead to prefetch—plays the role of **pipeline** depth: too short and the load does not complete before the data is needed, too long and the prefetched line is evicted before use or displaces something hot. Tuning that distance against the memory latency and the per-iteration compute is the **cpu** equivalent of choosing the number of buffers on a GPU.

The absence of a scratchpad changes what "double buffering" even means on a **cpu**. There is no second explicit buffer to fill; instead the two stages share the cache, and the prefetch distance determines how far ahead the next tile's lines are brought in before eviction pressure or the prefetcher's own heuristics stop it (Goodfellow *et al.*, 2016). This makes the **cpu pipeline** softer than a GPU's—there is no hard barrier guaranteeing buffer B is full, only the statistical likelihood that the prefetched lines arrived—so correctness is never at risk (a cache miss just stalls) but performance is more variable. The practical implication is that **cpu** decode tuning is validated empirically by measuring cache-miss rates and achieved bandwidth at the decode operating point, rather than by reasoning about a fixed **pipeline** depth as on a GPU.

Some server CPUs add a dedicated **dma**-style data-movement engine that can copy memory regions asynchronously and offload bulk transfers from the cores, which brings the **cpu** closer to the explicit asynchronous model of a GPU or NPU (Goodfellow *et al.*, 2016). Where available, such an engine lets a decode kernel stage a weight or KV block while the cores compute, an explicit **double-buffer** between memory tiers. But the dominant reality on a **cpu** is that residency and overlap are governed by the cache and prefetcher, so the **pipeline** is largely implicit; the multi-hardware contrast with the GPU is that the GPU programmer builds the pipeline explicitly with **TMA** and barriers, while the **cpu** programmer shapes access patterns and prefetch distance to coax the hardware into building an equivalent one (SemiAnalysis, 2024; Chen, 2026). For a decode kernel this means the highest-leverage **cpu** move is often simply ensuring the weight stream

is contiguous and sequentially accessed so the prefetcher tracks it perfectly, rather than hand-rolling an explicit **double-buffer** that the hardware would have built anyway (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). The compiler's job on a **cpu** is therefore as much about laying out data for predictable access as about scheduling explicit transfers.

1.4 NPU pipeline

On a spatial **npu** the **double-buffer pipeline** is the most explicit of all and is the native execution model. Each AMD XDNA / AIE-ML compute tile has a dedicated **dma** engine that moves data between streams and the tile's 64 KB local SRAM, and the architecture provides task queues and task-completion-tokens so the **dma** can run ahead of compute and signal when a buffer is ready (AMD, 2024a,b). The completion token is the **npu**'s version of the GPU's transaction barrier: the vector unit waits on the token before consuming a buffer, and the **dma** fills the other buffer in the meantime.

Because there is no cache to absorb a late load, double buffering is not an optimization on an **npu**—it is mandatory. A compute tile that waits on its **dma** simply idles, so the schedule ping-pongs between two buffers in the tile SRAM: load buffer B via the **dma** engine while the vector unit consumes buffer A, then swap, with the task-completion-tokens sequencing the swap (AMD, 2024a,b). This halves the usable tile SRAM for the double-buffered operand, which is the residency charge the planner of Chapter 6 must include, and it is why the tile shapes are sized so that two copies of the streamed operand plus the accumulators fit the 64 KB budget.

The AIE-ML **dma** is more capable than a simple copy engine, and those features map directly onto **pipeline** needs. It supports task queues, so multiple transfers can be enqueued and run back-to-back without per-transfer setup from the vector unit, and task-completion-tokens, so the consumer can wait on a specific transfer's completion rather than a coarse fence (AMD, 2024a). It also offers multidimensional addressing, which lets a single **dma** task gather a strided tile—a row of a weight matrix, say—without the vector unit computing addresses, the **npu** analogue of **TMA**'s multidimensional descriptor (AMD, 2024a,b). These features exist because on a fabric with no cache the **dma** must do all the work of keeping the compute fed, so it is engineered to run a deep, autonomous **pipeline** with minimal vector-unit involvement.

The spatial structure gives the **npu pipeline** a dimension the GPU and **cpu** lack: the stages can be different physical tiles. A producer tile can stream data

Table 1.1: Asynchronous-movement primitive and completion signal per hardware class; all build the same double-buffer pipeline (Shah *et al.*, 2024; AMD, 2024a).

Class	Async primitive	Completion signal
GPU (Hopper)	TMA bulk copy	Transaction (mbarrier)
CPU	Prefetch / DMA engine	Cache fill / engine status
NPU (XDNA)	Tile DMA engine	Task-completion-token

into a neighbor's local memory, which computes and forwards to the next tile, so a multi-stage **pipeline** is laid out across the array with each tile a fixed stage and the **dma**/stream interconnect carrying data between them (AMD, 2024b,a). This is the spatial analogue of a deep software **pipeline**: instead of one unit cycling through buffers, several units form an assembly line, and the on-chip **dma** keeps each stage fed. The decode payoff is the same as everywhere—intermediate results never leave on-chip SRAM, so only inputs and final outputs traverse DDR (Dao *et al.*, 2022; Chen, 2026).

The trade the spatial **pipeline** makes is depth for latency and balance. A pipeline spread across k tiles can keep k stages busy and hide more latency, but it also has a longer fill and drain, and it requires the per-stage work to be balanced so no tile becomes the bottleneck that starves the rest (AMD, 2024b,a). Because the assignment of stages to tiles is fixed at configuration time, the compiler must balance the stages statically—splitting a heavy stage across two tiles, or fusing two light stages onto one—which is the spatial counterpart to choosing a GPU **pipeline**'s depth. Getting this balance wrong does not merely slow the kernel; on a fabric with no cache fallback it idles whole tiles, so the **npu pipeline** demands the most careful static planning of the three classes, and rewards it with the most deterministic per-token cost (SemiAnalysis, 2024; Davies *et al.*, 2025).

1.5 YiRage pipeline generation

The book's reference compiler turns pipeline construction into a generated, per-target artifact. **YiRage** carries a **pipeline**-generation pass that, given a fused region and a target, emits the correct **double-buffer pipeline**: **TMA** producer warps plus transaction barriers on **Hopper**, prefetch-shaped access patterns on a **cpu**, and ping-pong **dma** with completion tokens on an XDNA **npu** (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Because the residency budget and the role assignment are already fixed by the passes of Chapters 6 and 7, the

pipeline pass knows how much fast memory is available for buffers and which units are producers, so it can choose the **pipeline** depth that fits and wire the completion signals correctly (Wu *et al.*, 2024; AMD, 2024a).

Pipeline depth is the parameter the pass tunes per target. On a **Hopper** GPU with a 228 KB shared-memory budget it can afford several **TMA** buffers in flight, hiding more latency; on a 64 KB XDNA tile it can usually afford only the mandatory two, so the **pipeline** is shallow and the tile shapes shrink accordingly (NVIDIA, 2022; AMD, 2024a). Getting this wrong in either direction is a measurable decode regression: too shallow and the compute units stall on loads, too deep and the buffers crowd out the working set and force a spill that reintroduces the HBM traffic the **pipeline** was meant to hide (Vellaisamy *et al.*, 2026; Chen, 2026). The pass therefore solves depth jointly with the residency check rather than applying a fixed constant.

The pass also has to reason about a subtlety the hardware exposes: the **pipeline** prologue and epilogue. The first $s-1$ tiles of an s-deep **pipeline** only fill buffers without producing output (the prologue), and the last $s-1$ steps only drain buffers without issuing new loads (the epilogue); these transient phases are not overlapped and their cost is fixed per kernel invocation (Shah *et al.*, 2024; Dao *et al.*, 2022). For a long-running prefill GEMM the prologue cost is negligible against thousands of steady-state steps, but for a short batch-1 decode kernel that processes few tiles per token the prologue can be a meaningful fraction of the runtime. This is precisely why persistent kernels from Chapter 7 matter here: by keeping the kernel resident across tokens, the **pipeline** stays warm and the prologue is paid once rather than per token, so movement pipelining and static residency reinforce each other (Shah *et al.*, 2024; Mirage Project contributors, 2025).

A second decision the pass makes is whether to pipeline the KV-cache stream separately from the weight stream. The two have different lifetimes and sizes—weights are reused across all positions of a layer while the KV slice grows with context—so a single **pipeline** depth is rarely optimal for both (Kwon *et al.*, 2023; Dao *et al.*, 2023). The pass can assign separate buffers and depths to each stream, deepening the KV **pipeline** when context is long and the KV read dominates, and the weight **pipeline** when the model is wide and weight streaming dominates. This per-stream tuning is the kind of decision that is tedious and error-prone by hand but mechanical for a compiler that knows the shapes and the residency budget, and it is where a generated **pipeline** can beat a hand-written one that uses a single uniform depth (Chen and YiRage Contributors, 2026; Chen, 2026).

What makes the **YiRage pipeline** pass composable is that it is the movement counterpart to the static role assignment of Chapter 7: roles decide which unit

does what, and the **pipeline** pass decides when each unit's data arrives (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Together they realize a fused region as a schedule-free, fully overlapped kernel, and they compose with the megakernel search of Chapter 20—the superoptimizer proposes the fusion, the memory pass certifies its footprint, the tiling pass assigns roles, and the **pipeline** pass overlaps the movement, all per target and all checked before codegen (Wu *et al.*, 2024; Mirage Project contributors, 2025). The result is that the asynchronous, double-buffered pipeline a CUDA expert once hand-wrote becomes a compiler output, regenerated correctly for each SKU in a heterogeneous fleet.

1.6 Key Takeaways

- **Overlap movement with compute via a double-buffer pipeline.** A synchronous load-then-compute loop exposes full memory latency every step; a double-buffer pipeline hides it behind the previous tile's compute after a one-tile prologue, and deepening the pipeline helps only until the bottleneck stage saturates (Dao *et al.*, 2022; SemiAnalysis, 2024). At batch-1 decode the load stage dominates, so overlap is the win (Vellaisamy *et al.*, 2026).

- **TMA is Hopper's asynchronous movement primitive.** Issued by a single thread (saving registers), TMA performs async bulk gmem→smem copies signaled by transaction barriers, with cluster multicast and L2 promotion hints (Shah *et al.*, 2024; NVIDIA, 2022). FlashAttention-3 pairs TMA producers with WGMMA consumers (SemiAnalysis, 2024).

- **CPU pipelines are prefetch- and cache-shaped.** With no scratchpad, a CPU overlaps through hardware prefetch, software prefetch distance (its pipeline depth), and sometimes a DMA engine; the kernel shapes access patterns so the hardware builds the pipeline (Goodfellow *et al.*, 2016; Chen, 2026). Residency is governed by the cache.

- **NPU double buffering is mandatory and explicit.** An XDNA tile's DMA engine ping-pongs two buffers in 64 KB SRAM with task-completion-tokens, and multi-stage pipelines span physical tiles across the array (AMD, 2024a,b). With no cache fallback on the fabric, a stalled DMA simply idles the whole compute tile.

- **Generate the pipeline per target.** The YiRage pipeline pass emits TMA+barriers, prefetch patterns, or ping-pong DMA per class and solves pipeline depth jointly with the residency budget, with separate depths for

the weight and KV streams (Chen and YiRage Contributors, 2026; NVIDIA, 2022). Too shallow stalls; too deep spills (Vellaisamy *et al.*, 2026).

1.7 Conclusion

Asynchronous movement is what keeps the statically assigned consumers of Chapter 7 from stalling: a double-buffer pipeline overlaps the load of the next tile with the compute on the current one, hiding the memory latency that otherwise dominates batch-1 decode (Shah *et al.*, 2024; Chen, 2026). Each hardware class provides the primitive—TMA with transaction barriers on Hopper, prefetch and DMA engines on a CPU, ping-pong tile DMA with completion tokens on an XDNA NPU—and the YiRage pipeline pass generates the correct, depth-tuned pipeline per target, with separate depths for the weight and KV streams where their sizes differ, composing with the residency, role, and search passes of the surrounding chapters (Chen and YiRage Contributors, 2026; AMD, 2024a; Wu *et al.*, 2024). A pipeline only works if its stages synchronize correctly, however, and at the scale of a megakernel spanning many units the synchronization itself becomes a design problem—which is the subject of Chapter **??**: hierarchical synchronization and communication across threads, warps, tiles, and devices (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 3, 7, 8, 9, 10, 11

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 7, 8, 10

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 6, 8, 9, 10, 11

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 8, 9, 10, 11

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 2, 9

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 3, 4, 5, 8, 9, 10

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 3, 8, 11

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 5, 6, 7, 10

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 5, 7, 9

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 9, 10

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 4, 5, 9, 10, 11

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 6, 8, 10

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMA), persistent kernels with tile scheduler, pingpong scheduling. 1, 2, 3, 4, 5, 8, 9, 10, 11

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 5, 9, 10, 11

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 9, 10, 11

Index

Asynchronous copy, 1

Double buffering, 1

TMA, 1